

Álgebra Lineal Numérica

Guía Completa de Computación Científica con numpy.linalg

Fundamentos Matemáticos, Estabilidad Numérica, Descomposiciones Matriciales y
Aplicaciones Prácticas



Autor: Laboratorio de Métodos Numéricos y Computación Científica
Referencia Base: *Semana 10 – Uso de la librería numpy.linalg en Python*
Versión: 2.0 (Edición Avanzada y Mejorada) | **Fecha:** 18 de septiembre de 2026

Navegación Interactiva: Este documento incorpora una **bolita flotante interactiva** en la esquina inferior derecha de cada página. Al hacer clic sobre ella en cualquier momento de la lectura, el visor de PDF saltará automáticamente al **Índice General**.

Resumen Ejecutivo

El presente documento constituye una versión profundamente enriquecida, formalizada y ampliada de la guía práctica sobre el submódulo `numpy.linalg` de la librería **NumPy** para Python. A diferencia de un compendio puramente sintáctico, esta obra fusiona la **teoría matemática rigurosa** del álgebra lineal con los **fundamentos computacionales de la aritmética de punto flotante** (estándar IEEE 754) y las rutinas subyacentes de **BLAS** (*Basic Linear Algebra Subprograms*) y **LAPACK** (*Linear Algebra Package*).

A lo largo de este manual se profundiza en las operaciones estructurales (determinantes, trazas, rangos, normas), la resolución de sistemas lineales compatibles y sobredeterminados, el análisis espectral de autovalores y autovectores, las factorizaciones matriciales fundamentales (**LU**, **QR**, **Cholesky**, **SVD**), el condicionamiento numérico y el cálculo de pseudoinversas. Cada sección incluye fundamentos teóricos, formulación matricial explícita, código en Python con análisis de salidas, consideraciones de complejidad asintótica $\mathcal{O}(\cdot)$, advertencias de estabilidad numérica y buenas prácticas profesionales en ingeniería de datos, física computacional y aprendizaje automático (*Machine Learning*).

Índice

Resumen Ejecutivo	1
1. Introducción y Arquitectura Numérica de NumPy	3
1.1. El Papel Central de <code>numpy.linalg</code>	3
1.2. Infraestructura Subyacente: BLAS y LAPACK	3
1.3. Convenciones de Importación	3
2. Determinante de una Matriz Cuadrada	4
2.1. Fundamentación Matemática	4
2.2. Implementación y Propiedades en Python	4
2.3. Estabilidad Numérica y el Peligro del Desbordamiento	4
3. Matriz Inversa y Reglas de Cómputo	5
3.1. Marco Teórico	5
3.2. Implementación y Manejo de Singularidad	5
3.3. ¿Por Qué Nunca se Debe Invertir una Matriz para Resolver $Ax = b$?	6
4. Resolución de Sistemas de Ecuaciones Lineales	6
4.1. Formulación Matricial y Algoritmo	6
4.2. Implementación y Demostración Comparativa	6
5. Autovalores y Autovectores (Análisis Espectral)	7
5.1. Definición y Teorema Espectral	7
5.2. <code>np.linalg.eig</code> vs <code>np.linalg.eigh</code>	8
6. Rango de una Matriz y Tolerancia Numérica	8
6.1. Definición y Cómputo por SVD	8
7. Traza de una Matriz y Propiedades Invariantes	9
7.1. Definición Algebraica	9
7.2. Teoremas y Propiedades de Invarianza Espectral	9

8. Normas Vectoriales y Matriciales	10
8.1. Axiomática de las Normas	10
8.2. Normas Vectoriales en <code>numpy.linalg.norm</code>	11
8.3. Normas Matriciales	11
9. Descomposición en Valores Singulares (SVD)	12
9.1. Teorema Fundamental de la SVD	12
9.2. SVD Reducida vs SVD Completa	12
9.3. Teorema de Eckart-Young-Mirsky y Reducción de Rango	12
10.Descomposición QR	13
10.1. Definición y Propiedades	13
10.2. Aplicación a Mínimos Cuadrados	13
11.Descomposición de Cholesky	14
11.1. Requisitos y Definición Matemática	14
11.2. Ventajas Computacionales	14
12.Pseudoinversa de Moore-Penrose	15
12.1. Las Cuatro Condiciones Axiomáticas	15
12.2. Conexión con la SVD	15
13.Mínimos Cuadrados Lineales (lstsq)	16
13.1. Formulación del Problema Sobredeterminado	16
14.Número de Condición y Sensibilidad Numérica	16
14.1. Definición Formal y Propagación de Errores	16
15.Tabla Resumen de Funciones y Complejidad	18
16.Ejemplo Integrador Completo Multidisciplinar	18
17.Decálogo de Buenas Prácticas en Álgebra Lineal	19
18.Conclusiones y Referencias Bibliográficas	20

Buena Práctica y Rendimiento: Uso de la Bolita de Navegación

En todas las páginas de contenido subsiguientes, observe el botón circular (▲ ÍNDICE) ubicado en la esquina inferior derecha. Al hacer clic en él, regresará inmediatamente a este índice.

1. Introducción y Arquitectura Numérica de NumPy

1.1. El Papel Central de `numpy.linalg`

El submódulo `numpy.linalg` conforma el núcleo computacional para álgebra lineal numérica en el ecosistema científico de Python. A diferencia de las implementaciones pedagógicas basadas en bucles nativos de Python —que adolecen de un costo por interpretación muy elevado y falta de aprovechamiento del caché del procesador—, `numpy.linalg` delega la totalidad del cómputo intensivo a librerías escritas en C y Fortran altamente optimizadas a nivel de registros vectoriales (SIMD: AVX-512, NEON) y multihilo.

1.2. Infraestructura Subyacente: BLAS y LAPACK

NumPy no reinventa los algoritmos de descomposición matricial; en su lugar, se compila enlazado dinámicamente con las bibliotecas estándar de la industria:

- **BLAS (Basic Linear Algebra Subprograms):** Clasificadas en tres niveles:
 - **Nivel 1:** Operaciones vector-vector como productos escalares y normas ($\mathcal{O}(n)$ operaciones con $\mathcal{O}(n)$ accesos a memoria).
 - **Nivel 2:** Operaciones matriz-vector como $y \leftarrow \alpha Ax + \beta y$ ($\mathcal{O}(n^2)$ operaciones con $\mathcal{O}(n^2)$ accesos).
 - **Nivel 3:** Operaciones matriz-matriz como $C \leftarrow \alpha AB + \beta C$ ($\mathcal{O}(n^3)$ operaciones con $\mathcal{O}(n^2)$ transferencias de datos, posibilitando una eficiencia de caché masiva mediante bloqueo por bloques).
- **LAPACK (Linear Algebra Package):** Proporciona rutinas avanzadas para resolución de sistemas lineales, mínimos cuadrados, cálculo de autovalores y descomposiciones espectrales (**LU**, **QR**, **SVD**, **Cholesky**). Dependiendo de la distribución (OpenBLAS, Intel MKL o Apple Accelerate), los algoritmos emplean paralelismo automático multicúcleo.

1.3. Convenciones de Importación

Existen dos modalidades principales de importación en proyectos de producción:

```

1 import numpy as np
2
3 # Modalidad estándar y universalmente aceptada en la comunidad científica:
4 # Las funciones se invocan explícitamente como np.linalg.<funcion>
5 A = np.array([[2.0, 1.0], [1.0, 3.0]], dtype=np.float64)
6 det_A = np.linalg.det(A)
7
8 # Alternativamente, para scripts con uso intensivo de álgebra lineal:
9 import numpy.linalg as la
10 det_A_alt = la.det(A)

```

Listing 1: Importación canónica de NumPy y su submódulo de álgebra lineal

Buena Práctica y Rendimiento: Disposición de Memoria: C-contiguous vs F-contiguous

Por defecto, NumPy crea arreglos con orden de almacenamiento por filas (*C-contiguous*). Sin embargo, muchas subrutinas internas de LAPACK están escritas históricamente en Fortran y requieren matrices ordenadas por columnas (*Fortran-contiguous*). Al transferir matrices a `numpy.linalg`, NumPy puede generar copias en memoria si el orden o el tipo de dato (`dtype`) no coincide exactamente con el requerido por la subrutina LAPACK subyacente (por ejemplo, `float64` para doble precisión en `dgesv`, `dgeev`).

2. Determinante de una Matriz Cuadrada

2.1. Fundamentación Matemática

Para una matriz cuadrada $\mathbf{A} \in \mathbb{R}^{n \times n}$, el determinante $\det(\mathbf{A})$ o $|\mathbf{A}|$ es un escalar que cuantifica la alteración de volumen inducida por la transformación lineal asociada:

Definición: Determinante

Formalmente, mediante la fórmula de permutaciones de Leibniz:

$$\det(\mathbf{A}) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{i=1}^n A_{i,\sigma(i)} \quad (1)$$

donde S_n representa el conjunto de las $n!$ permutaciones del conjunto $\{1, 2, \dots, n\}$ y $\text{sgn}(\sigma) \in \{+1, -1\}$ es el signo o paridad de la permutación.

Geométricamente, en $\mathbb{R}^2$ el determinante corresponde al área con signo del paralelogramo generado por los vectores columna; en $\mathbb{R}^3$, al volumen orientado del paralelepípedo. Un determinante nulo implica que la transformación proyecta el espacio en una dimensión inferior, colapsando el volumen a cero.

2.2. Implementación y Propiedades en Python

```

1 import numpy as np
2
3 # Matriz 2x2
4 A = np.array([[4.0, 7.0],
5               [2.0, 6.0]])
6 det_A = np.linalg.det(A)
7 print(f"det(A) = {det_A:.4f}") # 4*6 - 7*2 = 24 - 14 = 10.0
8
9 # Matriz 3x3
10 B = np.array([[1.0, 2.0, 3.0],
11               [4.0, 5.0, 6.0],
12               [7.0, 8.0, 10.0]])
13 det_B = np.linalg.det(B)
14 print(f"det(B) = {det_B:.4f}") # -3.0
15
16 # Propiedad multiplicativa: det(A @ C) == det(A) * det(C)
17 C = np.array([[2.0, 0.0],
18               [1.0, 5.0]])
19 print("det(A @ C):", np.linalg.det(A @ C))
20 print("det(A)*det(C):", np.linalg.det(A) * np.linalg.det(C))

```

Listing 2: Cálculo de determinantes con `np.linalg.det`

2.3. Estabilidad Numérica y el Peligro del Desbordamiento

Advertencia Numérica: Desbordamiento y Subdesbordamiento en Determinantes Grandes

El cálculo explícito del determinante mediante $\prod_{i=1}^n \lambda_i$ o mediante la diagonal de la factorización **PLU** escala de forma exponencial en magnitud. Si una matriz de 100×100 tiene valores propios del orden de 10, su determinante será 10^{100} , provocando un desbordamiento de punto flotante (*overflow* a `inf`) o *underflow* a `0.0`.

Solución Profesional: En aplicaciones estadísticas avanzadas (como modelos gaussianos mul-

tivariados o filtros de partículas), utilice siempre `np.linalg.slogdet(A)`, que computa el signo y el logaritmo natural del valor absoluto del determinante de forma numéricamente protegida:

$$\ln |\det(\mathbf{A})| = \sum_{i=1}^n \ln |U_{ii}| \quad (2)$$

```
1 # Matriz de dimension 60x60 escalada
2 M = np.eye(60) * 10.0
3 signo, logdet = np.linalg.slogdet(M)
4 print(f"Signo: {signo}, Log-Determinante: {logdet:.4f}")
5 # det(M) teorico = 10^60. En logaritmo natural: 60 * ln(10) = 138.1551
```

Listing 3: Uso de `np.linalg.slogdet` para evitar problemas de escala

3. Matriz Inversa y Reglas de Cómputo

3.1. Marco Teórico

Una matriz cuadrada $\mathbf{A} \in \mathbb{R}^{n \times n}$ es **invertible** (o regular / no singular) si y sólo si existe una matriz única $\mathbf{A}^{-1} \in \mathbb{R}^{n \times n}$ tal que:

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_n \quad (3)$$

Condiciones equivalentes para la invertibilidad (Teorema Fundamental de Matrices Invertibles):

1. $\det(\mathbf{A}) \neq 0$.
2. El rango de $\mathbf{A}$ es máximo: $\text{rango}(\mathbf{A}) = n$.
3. Las columnas (y filas) de $\mathbf{A}$ forman una base linealmente independiente en $\mathbb{R}^n$.
4. Ningún autovalor de $\mathbf{A}$ es idénticamente cero ($\lambda_i \neq 0 \forall i$).
5. El núcleo o espacio nulo es trivial: $\ker(\mathbf{A}) = \{\mathbf{0}\}$.

3.2. Implementación y Manejo de Singularidad

```
1 import numpy as np
2
3 A = np.array([[4.0, 7.0],
4               [2.0, 6.0]])
5
6 # Calculo de la inversa analitica y numerica
7 inv_A = np.linalg.inv(A)
8 print("Inversa computada:\n", inv_A)
9
10 # Verificacion numerica de la identidad: A @ inv_A = I
11 identidad_aprox = A @ inv_A
12 print("Comprobacion A @ A^(-1):\n", np.round(identidad_aprox, 6))
13
14 # Uso correcto de comparacion con tolerancia de maquina
15 print("Es identica a I?", np.allclose(identidad_aprox, np.eye(2)))
16
17 # Manejo de excepciones en matrices singulares (det = 0)
18 S = np.array([[2.0, 4.0],
19               [1.0, 2.0]]) # Fila 1 es el doble de la fila 2
20
21 try:
```

```

22     inv_S = np.linalg.inv(S)
23 except np.linalg.LinAlgError as e:
24     print(f"Error capturado con éxito: {e}")

```

Listing 4: Inversión de matrices y control de excepciones

3.3. ¿Por Qué Nunca se Debe Invertir una Matriz para Resolver $Ax = b$?

Advertencia Numérica: Regla de Oro del Álgebra Lineal Numérica

En libros de texto de matemáticas elementales, la solución teórica del sistema $\mathbf{Ax} = \mathbf{b}$ se formula como $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$. **¡Bajo ninguna circunstancia implemente esto en código numérico real!**

- **Costo Computacional:** Invertir una matriz de $n \times n$ requiere aproximadamente $2n^3$ operaciones de punto flotante (*flops*). En contraste, la factorización **LU** con sustitución progresiva y regresiva requiere únicamente $\frac{2}{3}n^3$ *flops* (un tercio del costo).
- **Estabilidad Numérica:** El cálculo explícito de $\mathbf{A}^{-1}$ introduce errores de redondeo innecesarios. Cuando la matriz posee un número de condición elevado, multiplicar $\mathbf{A}^{-1}\mathbf{b}$ magnifica el error relativo de forma catastrófica frente a la resolución directa por eliminación gaussiana con pivoteo parcial.

4. Resolución de Sistemas de Ecuaciones Lineales

4.1. Formulación Matricial y Algoritmo

Un sistema de n ecuaciones algebraicas lineales con n incógnitas se escribe matricialmente como:

$$\mathbf{Ax} = \mathbf{b} \iff \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} \quad (4)$$

La función `np.linalg.solve(A, b)` ejecuta la subrutina `dgesv` de LAPACK, la cual efectúa una factorización con pivoteo parcial:

$$\mathbf{PA} = \mathbf{LU} \quad (5)$$

donde **P** es una matriz de permutación de filas, **L** es triangular inferior unitaria y **U** es triangular superior. La solución se obtiene resolviendo secuencialmente:

1. $\mathbf{Ly} = \mathbf{Pb}$ mediante sustitución hacia adelante (*forward substitution*).
2. $\mathbf{Ux} = \mathbf{y}$ mediante sustitución hacia atrás (*back substitution*).

4.2. Implementación y Demostración Comparativa

```

1 import numpy as np
2 import time
3
4 # Sistema:
5 # 3x + 2y - z = 1
6 # 2x - 2y + 4z = -2
7 # -x + 0.5y - z = 0
8 A = np.array([[ 3.0,  2.0, -1.0],
9               [ 2.0, -2.0,  4.0],

```

```

10         [-1.0, 0.5, -1.0]])
11 b = np.array([1.0, -2.0, 0.0])
12
13 # Resolución profesional con solve():
14 x = np.linalg.solve(A, b)
15 print("Vector solución x:", x)
16
17 # Cálculo del residuo euclidiano: ||A x - b||_2
18 residuo = np.linalg.norm(A @ x - b)
19 print(f"Norma del residuo ||Ax - b||: {residuo:.2e}")
20
21 # Comparación empírica de tiempo y precisión para matriz grande
22 N = 1200
23 np.random.seed(42)
24 M = np.random.randn(N, N) + N * np.eye(N) # Diagonalmente dominante
25 rhs = np.random.randn(N)
26
27 # Metodo 1: solve
28 t0 = time.perf_counter()
29 sol_solve = np.linalg.solve(M, rhs)
30 t_solve = time.perf_counter() - t0
31
32 # Metodo 2: inv @ rhs
33 t0 = time.perf_counter()
34 sol_inv = np.linalg.inv(M) @ rhs
35 t_inv = time.perf_counter() - t0
36
37 print(f"\nTiempo solve(): {t_solve*1000:.2f} ms")
38 print(f"Tiempo inv() @ b: {t_inv*1000:.2f} ms ({t_inv/t_solve:.2f})x mas lento")
39 print("Diferencia máxima absoluta entre soluciones:", np.max(np.abs(sol_solve -
    sol_inv)))

```

Listing 5: Resolución de sistemas con `np.linalg.solve` y comparación con la inversa

Buena Práctica y Rendimiento: Resolución con Múltiples Vectores de Términos Independientes

Si se tienen varios sistemas con la misma matriz de coeficientes $\mathbf{A}\mathbf{X} = \mathbf{B}$, donde $\mathbf{B} \in \mathbb{R}^{n \times k}$, pásese directamente la matriz $\mathbf{B}$ a `np.linalg.solve(A, B)`. La factorización **PLU** se computará una única vez y se reutilizará para las k columnas.

5. Autovalores y Autovectores (Análisis Espectral)

5.1. Definición y Teorema Espectral

Para una transformación lineal representada por $\mathbf{A} \in \mathbb{R}^{n \times n}$, un escalar $\lambda \in \mathbb{C}$ es un **autovalor** (o valor propio) y un vector no nulo $\mathbf{v} \in \mathbb{C}^n$ es su correspondiente **autovector** si satisfacen:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v} \iff (\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0} \quad (6)$$

Los autovalores son las raíces del polinomio característico:

$$p(\lambda) = \det(\mathbf{A} - \lambda\mathbf{I}) = 0 \quad (7)$$

Teorema y Propiedad: Teorema Espectral para Matrices Simétricas Reales

Si $\mathbf{A} \in \mathbb{R}^{n \times n}$ es simétrica ($\mathbf{A} = \mathbf{A}^T$), entonces:

1. Todos sus autovalores son números estrictamente reales: $\lambda_i \in \mathbb{R}$.

2. Los autovectores correspondientes a autovalores distintos son mutuamente ortogonales.
3. Existe una base ortonormal completa de autovectores, permitiendo la descomposición ortogonal:

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T \quad \text{con} \quad \mathbf{V}^T\mathbf{V} = \mathbf{I}_n \quad (8)$$

5.2. np.linalg.eig vs np.linalg.eigh

Tabla 1: Comparativa fundamental de funciones espectrales en NumPy

Función	Tipo de Matriz Aceptada	Garantía Matemática	Rutina LAPACK
eig(A)	Matriz cuadrada arbitraria	Autovalores y autovectores en $\mathbb{C}$	dgeev
eigh(A)	Simétrica real o Hermítica	Autovalores reales y ordenados, vectores ortonormales	dsyevd

```

1 import numpy as np
2
3 # Caso 1: Matriz general con posibles autovalores complejos o reales
4 A = np.array([[0.0, -1.0],
5               [1.0,  0.0]]) # Matriz de rotacion pura de 90 grados
6 w, v = np.linalg.eig(A)
7 print("Autovalores de matriz de rotacion (complejos):", w)
8 # Salida: [0.+1.j, 0.-1.j] (los valores propios son +i y -i)
9
10 # Caso 2: Matriz simetrica (recomendado SIEMPRE eigh)
11 S = np.array([[5.0, 2.0, 1.0],
12               [2.0, 4.0, 0.5],
13               [1.0, 0.5, 3.0]])
14
15 # eigh devuelve autovalores ordenados en forma ascendente: lambda_1 <= ... <=
16 # lambda_n
17 vals, vecs = np.linalg.eigh(S)
18 print("\nAutovalores reales ordenados:\n", vals)
19 print("Base ortonormal de autovectores (columnas):\n", vecs)
20
21 # Verificacion de ortonormalidad: V^T @ V = I
22 print("V^T @ V es la identidad?", np.allclose(vecs.T @ vecs, np.eye(3)))
23
24 # Verificacion espectral: A @ v_0 == lambda_0 * v_0
25 v0 = vecs[:, 0]
26 l0 = vals[0]
27 print("A @ v0 coincide con l0 * v0?", np.allclose(S @ v0, l0 * v0))

```

Listing 6: Uso de eig y eigh en matrices generales y simétricas

6. Rango de una Matriz y Tolerancia Numérica

6.1. Definición y Cómputo por SVD

El **rango** de una matriz $\mathbf{A} \in \mathbb{R}^{m \times n}$ es el número máximo de columnas (o filas) linealmente independientes, lo que equivale a la dimensión del espacio imagen:

$$\text{rango}(\mathbf{A}) = \dim(\text{col}(\mathbf{A})) \quad (9)$$

En matemática pura con aritmética exacta, el rango se evalúa mediante eliminación gaussiana contando pivotes no nulos. Sin embargo, en computación numérica de punto flotante, los errores de

redondeo casi siempre impiden que un pivote sea estrictamente 0,0, tomando en su lugar valores como $1,38 \times 10^{-16}$.

Por este motivo, `np.linalg.matrix_rank(A)` emplea la **Descomposición en Valores Singulares (SVD)**. El rango numérico se define como el número de valores singulares σ_i que superan un umbral de tolerancia τ :

$$\text{rango}(\mathbf{A}) = \#\{\sigma_i \mid \sigma_i > \tau\} \quad (10)$$

donde la tolerancia por defecto en NumPy es:

$$\tau = \sigma_{\text{máx}} \cdot \text{máx}(m, n) \cdot \epsilon_{\text{mach}} \quad (11)$$

siendo $\epsilon_{\text{mach}} \approx 2,22 \times 10^{-16}$ en doble precisión (`float64`).

```

1 import numpy as np
2
3 # Matriz 3x3 donde la tercera fila es exactamente la suma de las dos primeras
4 A = np.array([[1.0, 2.0, 3.0],
5               [4.0, 5.0, 6.0],
6               [5.0, 7.0, 9.0]])
7
8 r = np.linalg.matrix_rank(A)
9 print(f"Rango computado de A: {r}") # Esperado: 2
10
11 # Demostracion de estabilidad frente a perturbacion numerica pequena
12 A_ruidosa = A.copy()
13 A_ruidosa[2, 2] += 1e-15 # Perturbacion cercana al limite de precision maquina
14
15 r_robusto = np.linalg.matrix_rank(A_ruidosa)
16 print(f"Rango de matriz perturbada con tolerancia automatica: {r_robusto}")
17
18 # Valores singulares para comprender la decision del umbral:
19 _, s, _ = np.linalg.svd(A)
20 print("Valores singulares de A:", s)
21 # El menor valor singular sera del orden de 10^-16, por debajo del umbral tau

```

Listing 7: Evaluación robusta del rango matricial

7. Traza de una Matriz y Propiedades Invariantes

7.1. Definición Algebraica

La **traza** de una matriz cuadrada $\mathbf{A} \in \mathbb{R}^{n \times n}$ es la suma de los elementos ubicados en su diagonal principal:

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^n A_{ii} \quad (12)$$

En NumPy, la traza se evalúa directamente con `np.trace(A)` (o su método equivalente `A.trace()`).

7.2. Teoremas y Propiedades de Invarianza Espectral

Teorema y Propiedad: Propiedades Fundamentales de la Traza

Dadas matrices compatibles y escalares $\alpha, \beta \in \mathbb{R}$:

1. **Linealidad:** $\text{tr}(\alpha \mathbf{A} + \beta \mathbf{B}) = \alpha \text{tr}(\mathbf{A}) + \beta \text{tr}(\mathbf{B})$.
2. **Invarianza ante la transposición:** $\text{tr}(\mathbf{A}^T) = \text{tr}(\mathbf{A})$.

3. Propiedad Cíclica: Para matrices compatibles $\mathbf{A} \in \mathbb{R}^{m \times n}$ y $\mathbf{B} \in \mathbb{R}^{n \times m}$:

$$\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA}) \quad (13)$$

Generalizable a productos de más factores:

$$\text{tr}(\mathbf{ABC}) = \text{tr}(\mathbf{BCA}) = \text{tr}(\mathbf{CAB})$$

(nótese que en general $\text{tr}(\mathbf{ABC}) \neq \text{tr}(\mathbf{ACB})$).

4. Conexión con los Autovalores: La traza es idéntica a la suma de todos los autovalores de la matriz:

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^n \lambda_i \quad (14)$$

Por tanto, la traza es un invariante de semejanza matricial: $\text{tr}(\mathbf{P}^{-1}\mathbf{A}\mathbf{P}) = \text{tr}(\mathbf{A})$.

```

1 import numpy as np
2
3 A = np.array([[2.0, 5.0, 1.0],
4               [3.0, 7.0, 4.0],
5               [0.0, 2.0, 1.0]])
6
7 # Calculo directo
8 tr_A = np.trace(A)
9 print(f"Traza directa de A: {tr_A}") # 2 + 7 + 1 = 10.0
10
11 # Verificacion del teorema espectral: tr(A) == sum(lambdas)
12 autovalores = np.linalg.eigvals(A)
13 suma_lambdas = np.sum(autovalores).real
14 print(f"Suma de autovalores: {suma_lambdas:.4f}")
15 print("Coinciden?", np.isclose(tr_A, suma_lambdas))
16
17 # Verificacion de la propiedad ciclica tr(AB) == tr(BA)
18 B = np.array([[1.0, 0.0, 2.0],
19               [2.0, 1.0, 0.0],
20               [0.0, 3.0, 4.0]])
21 print("tr(AB):", np.trace(A @ B))
22 print("tr(BA):", np.trace(B @ A))
23 print("Es identica la traza ciclica?", np.isclose(np.trace(A @ B), np.trace(B @ A)))

```

Listing 8: Demostración computacional de las propiedades de la traza

8. Normas Vectoriales y Matriciales

8.1. Axiomática de las Normas

Una norma $\|\cdot\|$ sobre un espacio vectorial V sobre $\mathbb{R}$ es una función $V \rightarrow [0, \infty)$ que satisface para todo $\mathbf{x}, \mathbf{y} \in V$ y $\alpha \in \mathbb{R}$:

1. **No negatividad y separación:** $\|\mathbf{x}\| \geq 0$, y $\|\mathbf{x}\| = 0 \iff \mathbf{x} = \mathbf{0}$.
2. **Homogeneidad absoluta:** $\|\alpha\mathbf{x}\| = |\alpha|\|\mathbf{x}\|$.
3. **Desigualdad triangular:** $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$.

8.2. Normas Vectoriales en `numpy.linalg.norm`

Dada la señal o vector $\mathbf{x} = (x_1, \dots, x_n)^T \in \mathbb{R}^n$:

- **Norma L_1 (Manhattan / Taxicab):** $\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$. Promueve soluciones dispersas (*sparsity*) en regularización Lasso.
- **Norma L_2 (Euclidiana):** $\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$. Medida canónica de distancia física y energía.
- **Norma L_∞ (Uniforme o de Chebyshev):** $\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|$.
- **Norma L_p General:** $\|\mathbf{x}\|_p = (\sum_{i=1}^n |x_i|^p)^{1/p}$.

8.3. Normas Matriciales

Para matrices $\mathbf{A} \in \mathbb{R}^{m \times n}$, se distinguen dos familias fundamentales:

1. **Norma de Frobenius:** Considera la matriz como un vector aplanado de dimensión mn :

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2} = \sqrt{\text{tr}(\mathbf{A}^T \mathbf{A})} = \sqrt{\sum_{i=1}^{\min(m,n)} \sigma_i^2} \quad (15)$$

2. **Normas Inducidas por Operador:** Definen la máxima amplificación sobre vectores:

$$\|\mathbf{A}\|_p = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbf{A}\mathbf{x}\|_p}{\|\mathbf{x}\|_p} \quad (16)$$

- Para $p = 1$: Máxima suma absoluta por columnas: $\|\mathbf{A}\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|$.
- Para $p = \infty$: Máxima suma absoluta por filas: $\|\mathbf{A}\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|$.
- Para $p = 2$ (Norma Espectral): $\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A})$ (mayor valor singular).

```

1 import numpy as np
2
3 # Normas vectoriales
4 v = np.array([3.0, -4.0])
5 print("Norma L1 (suma abs):", np.linalg.norm(v, ord=1))      # 3 + 4 = 7.0
6 print("Norma L2 (euclidiana):", np.linalg.norm(v, ord=2))    # sqrt(9+16) = 5.0
7 print("Norma Linf (max abs):", np.linalg.norm(v, ord=np.inf)) # max(3, 4) = 4.0
8
9 # Normas matriciales
10 M = np.array([[1.0, -2.0],
11               [3.0, 4.0]])
12
13 print("\nNorma Frobenius ||M||_F:", np.linalg.norm(M, ord='fro'))
14 # sqrt(1 + 4 + 9 + 16) = sqrt(30) approx 5.4772
15
16 print("Norma 1 (max suma col):", np.linalg.norm(M, ord=1))    # max(1+3, 2+4) =
17     6.0
18 print("Norma inf (max suma fila):", np.linalg.norm(M, ord=np.inf)) # max(1+2, 3+4)
19     = 7.0
20 print("Norma 2 (espectral sigma_max):", np.linalg.norm(M, ord=2))

```

Listing 9: Cálculo integral de normas vectoriales y matriciales

9. Descomposición en Valores Singulares (SVD)

9.1. Teorema Fundamental de la SVD

La **Descomposición en Valores Singulares** es considerada el pináculo del álgebra lineal numérica moderna debido a que existe y es computable para **cualquier** matriz rectangular o cuadrada $\mathbf{A} \in \mathbb{R}^{m \times n}$.

Teorema y Propiedad: Descomposición SVD

Toda matriz $\mathbf{A} \in \mathbb{R}^{m \times n}$ admite una factorización de la forma:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \quad (17)$$

donde:

- $\mathbf{U} \in \mathbb{R}^{m \times m}$ es una matriz ortogonal ($\mathbf{U}^T\mathbf{U} = \mathbf{I}_m$) cuyas columnas son los **vectores singulares izquierdos** (autovectores de $\mathbf{A}\mathbf{A}^T$).
- $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ es una matriz pseudo-diagonal cuyos elementos diagonales $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(m,n)} \geq 0$ son los **valores singulares** (raíces cuadradas de los autovalores no negativos de $\mathbf{A}^T\mathbf{A}$).
- $\mathbf{V} \in \mathbb{R}^{n \times n}$ es una matriz ortogonal ($\mathbf{V}^T\mathbf{V} = \mathbf{I}_n$) cuyas columnas son los **vectores singulares derechos** (autovectores de $\mathbf{A}^T\mathbf{A}$). En la fórmula interviene transpuesta como $\mathbf{V}^T$.

9.2. SVD Reducida vs SVD Completa

En aplicaciones con matrices muy alargadas ($m \gg n$), la SVD completa calcula $\mathbf{U}$ de tamaño $m \times m$, lo que desperdicia memoria innecesaria. El parámetro `full_matrices=False` computa la **SVD reducida o económica**:

- $\mathbf{U}$ tiene dimensión $m \times k$ donde $k = \min(m, n)$.
- $\mathbf{\Sigma}$ se representa como un vector unidimensional de k valores singulares.
- $\mathbf{V}^T$ tiene dimensión $k \times n$.

9.3. Teorema de Eckart-Young-Mirsky y Reducción de Rango

Teorema y Propiedad: Aproximación Óptima de Bajo Rango

La mejor aproximación de rango $k < \text{rango}(\mathbf{A})$ a la matriz $\mathbf{A}$ en norma espectral $\|\cdot\|_2$ y norma de Frobenius $\|\cdot\|_F$ viene dada por truncar la suma externa de la SVD a los primeros k términos:

$$\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T \quad (18)$$

El error de aproximación exacto en norma 2 es el primer valor singular descartado: $\|\mathbf{A} - \mathbf{A}_k\|_2 = \sigma_{k+1}$.

```
1 import numpy as np
2
3 # Matriz 3x4 (m=3, n=4)
4 A = np.array([[1.0, 2.0, 3.0, 4.0],
```

```

5         [5.0, 6.0, 7.0, 8.0],
6         [9.0, 8.0, 7.0, 6.0]])
7
8 # Descomposicion SVD reducida
9 U, s, Vt = np.linalg.svd(A, full_matrices=False)
10 print("Dimensiones U:", U.shape)      # (3, 3)
11 print("Valores singulares s:", s)      # Longitud 3
12 print("Dimensiones Vt:", Vt.shape)     # (3, 4)
13
14 # Reconstruccion exacta original: A = U @ diag(s) @ Vt
15 A_reconstruida = U @ np.diag(s) @ Vt
16 print("Reconstruccion identica?", np.allclose(A, A_reconstruida))
17
18 # Compresion de datos: Aproximacion optima de rango 1 (k=1)
19 k = 1
20 A_k1 = s[0] * np.outer(U[:, 0], Vt[0, :])
21 error_frob = np.linalg.norm(A - A_k1, 'fro')
22 print(f"Error de aproximacion con rango 1: {error_frob:.4f}")
23 print(f"Varianza retenida: {(s[0]**2) / np.sum(s**2) * 100:.2f}%")

```

Listing 10: SVD completa, reconstrucción y aproximación de bajo rango

10. Descomposición QR

10.1. Definición y Propiedades

La factorización **QR** descompone una matriz rectangular $\mathbf{A} \in \mathbb{R}^{m \times n}$ (con $m \geq n$) en el producto de:

$$\mathbf{A} = \mathbf{QR} \quad (19)$$

donde:

- $\mathbf{Q} \in \mathbb{R}^{m \times n}$ tiene columnas ortonormales: $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}_n$.
- $\mathbf{R} \in \mathbb{R}^{n \times n}$ es una matriz **triangular superior** invertible (si $\mathbf{A}$ tiene rango completo).

NumPy ejecuta la subrutina `dgeqrf` de LAPACK, basada en **reflectores de Householder**, superando en estabilidad numérica a la ortogonalización clásica de Gram-Schmidt.

10.2. Aplicación a Mínimos Cuadrados

Para resolver el problema sobredeterminado $\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2$:

$$\mathbf{Ax} = \mathbf{b} \iff \mathbf{QRx} = \mathbf{b} \iff \mathbf{Rx} = \mathbf{Q}^T \mathbf{b} \quad (20)$$

Dado que $\mathbf{R}$ es triangular superior, el sistema se resuelve inmediatamente mediante sustitución regresiva sin necesidad de formar $\mathbf{A}^T \mathbf{A}$, evitando cuadrar el número de condición ($\kappa(\mathbf{A}^T \mathbf{A}) = \kappa(\mathbf{A})^2$).

```

1 import numpy as np
2
3 A = np.array([[1.0, -1.0],
4               [1.0,  0.0],
5               [1.0,  1.0]])
6 b = np.array([2.0, 3.0, 5.0])
7
8 # Factorizacion QR reducida
9 Q, R = np.linalg.qr(A)
10 print("Matriz Q (columnas ortonormales):\n", Q)
11 print("Matriz R (triangular superior):\n", R)

```

```

12
13 # Verificacion ortonormal: Q^T @ Q = I
14 print("Q^T @ Q es I?", np.allclose(Q.T @ Q, np.eye(2)))
15
16 # Solucion de minimos cuadrados via QR: R x = Q^T b
17 x_qr = np.linalg.solve(R, Q.T @ b)
18 print("Solucion de minimos cuadrados x:", x_qr)

```

Listing 11: Factorización QR y verificación ortogonal

11. Descomposición de Cholesky

11.1. Requisitos y Definición Matemática

La descomposición de Cholesky es una factorización altamente eficiente aplicable exclusivamente a matrices simétricas y **definidas positivas** (SPD, por sus siglas en inglés: *Symmetric Positive Definite*):

Definición: Matriz Definida Positiva

Una matriz simétrica $\mathbf{A} \in \mathbb{R}^{n \times n}$ es definida positiva si:

$$\mathbf{x}^T \mathbf{A} \mathbf{x} > 0 \quad \forall \mathbf{x} \in \mathbb{R}^n, \mathbf{x} \neq \mathbf{0} \quad (21)$$

Condiciones equivalentes: todos sus autovalores son estrictamente positivos ($\lambda_i > 0$) y todos sus menores principales son estrictamente positivos (Criterio de Sylvester).

Teorema y Propiedad: Teorema de Cholesky

Si $\mathbf{A}$ es simétrica definida positiva, existe una única matriz triangular inferior $\mathbf{L}$ con elementos diagonales estrictamente positivos ($L_{ii} > 0$) tal que:

$$\mathbf{A} = \mathbf{L} \mathbf{L}^T \quad (22)$$

11.2. Ventajas Computacionales

- **Costo:** Requiere $\frac{1}{3}n^3$ flops (la mitad de operaciones que la factorización $\mathbf{LU}$ y no necesita almacenar la matriz $\mathbf{U}$).
- **Estabilidad incondicional:** No requiere ninguna estrategia de pivoteo de filas; el algoritmo es intrínsecamente estable frente a errores de redondeo.

```

1 import numpy as np
2
3 # Matriz simetrica definida positiva (SPD)
4 A = np.array([[4.0, 2.0],
5               [2.0, 5.0]])
6
7 # Descomposicion con cholesky
8 L = np.linalg.cholesky(A)
9 print("Factor triangular inferior L:\n", L)
10
11 # Verificacion: L @ L^T = A
12 print("L @ L^T recupera A?", np.allclose(L @ L.T, A))
13
14 # Aplicacion practica: Muestreo de vector aleatorio con matriz de covarianza A
15 # Si z ~ N(0, I), entonces x = L @ z ~ N(0, L @ L^T) = N(0, A)

```

```

16 np.random.seed(123)
17 z = np.random.randn(2, 50000)
18 x = L @ z
19 cov_empirica = np.cov(x)
20 print("\nMatriz de covarianza empirica simulada:\n", np.round(cov_empirica, 2))

```

Listing 12: Factorización de Cholesky y simulación de variables correlacionadas

12. Pseudoinversa de Moore-Penrose

12.1. Las Cuatro Condiciones Axiomáticas

Para una matriz arbitraria $\mathbf{A} \in \mathbb{R}^{m \times n}$ (que puede ser no cuadrada, de rango deficiente o singular), existe una matriz única $\mathbf{A}^+ \in \mathbb{R}^{n \times m}$, denominada **pseudoinversa de Moore-Penrose**, caracterizada por las 4 ecuaciones:

1. $\mathbf{A}\mathbf{A}^+\mathbf{A} = \mathbf{A}$
2. $\mathbf{A}^+\mathbf{A}\mathbf{A}^+ = \mathbf{A}^+$
3. $(\mathbf{A}\mathbf{A}^+)^T = \mathbf{A}\mathbf{A}^+$ (es un proyector ortogonal sobre $\text{col}(\mathbf{A})$)
4. $(\mathbf{A}^+\mathbf{A})^T = \mathbf{A}^+\mathbf{A}$ (es un proyector ortogonal sobre $\text{col}(\mathbf{A}^T)$)

12.2. Conexión con la SVD

La pseudoinversa se computa mediante la SVD: si $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, entonces:

$$\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^T \quad (23)$$

donde $\mathbf{\Sigma}^+$ se obtiene transponiendo $\mathbf{\Sigma}$ y tomando el recíproco $1/\sigma_i$ únicamente de los valores singulares que superan la tolerancia numérica, asignando 0 a los restantes.

```

1 import numpy as np
2
3 # Matriz rectangular 3x2 (sistema sobredeterminado)
4 A = np.array([[1.0, 2.0],
5               [3.0, 4.0],
6               [5.0, 6.0]])
7
8 pinv_A = np.linalg.pinv(A)
9 print("Pseudoinversa A^+ (dimension 2x3):\n", np.round(pinv_A, 4))
10
11 # Verificacion de las 4 condiciones de Moore-Penrose
12 cond1 = np.allclose(A @ pinv_A @ A, A)
13 cond2 = np.allclose(pinv_A @ A @ pinv_A, pinv_A)
14 cond3 = np.allclose((A @ pinv_A).T, A @ pinv_A)
15 cond4 = np.allclose((pinv_A @ A).T, pinv_A @ A)
16
17 print(f"Condiciones cumplidas: 1={cond1}, 2={cond2}, 3={cond3}, 4={cond4}")
18
19 # Si A tiene columnas linealmente independientes: pinv(A) == (A^T A)^(-1) A^T
20 analitica = np.linalg.inv(A.T @ A) @ A.T
21 print("Coincide con formula analitica?", np.allclose(pinv_A, analitica))

```

Listing 13: Pseudoinversa con `np.linalg.pinv` y comprobación de axiomas

13. Mínimos Cuadrados Lineales (lstsq)

13.1. Formulación del Problema Sobredeterminado

Cuando se dispone de más ecuaciones que incógnitas ($m > n$), el sistema $\mathbf{Ax} = \mathbf{b}$ carece en general de solución exacta. El método de mínimos cuadrados halla el vector $\hat{\mathbf{x}}$ que minimiza la suma de residuos al cuadrado:

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|_2^2 \quad (24)$$

La función `np.linalg.lstsq(A, b, rcond=None)` resuelve este problema con máxima estabilidad empleando SVD o QR. Retorna una tupla con 4 elementos:

- **x**: El vector solución óptimo de mínimos cuadrados $\hat{\mathbf{x}}$.
- **residuals**: La suma de los residuos al cuadrado $\|\mathbf{A}\hat{\mathbf{x}} - \mathbf{b}\|_2^2$ (sólo si $m > n$ y $\text{rango}(\mathbf{A}) = n$).
- **rank**: El rango numérico de la matriz $\mathbf{A}$.
- **s**: Los valores singulares de $\mathbf{A}$.

```

1 import numpy as np
2
3 # Datos experimentales sinteticos con ruido gaussiano
4 np.random.seed(42)
5 t = np.linspace(0, 5, 20)
6 # Modelo fisico real: y = 2.5 * t^2 - 1.2 * t + 3.0 + ruido
7 y_real = 2.5 * t**2 - 1.2 * t + 3.0
8 ruido = np.random.normal(0, 0.8, size=len(t))
9 y_obs = y_real + ruido
10
11 # Construccion de la matriz de diseno de Vandermonde: [t^2, t, 1]
12 A_diseno = np.column_stack([t**2, t, np.ones_like(t)])
13
14 # Ajuste por minimos cuadrados
15 coefs, res, rank, s = np.linalg.lstsq(A_diseno, y_obs, rcond=None)
16
17 print(f"Coeficientes estimados [a, b, c]: {coefs}")
18 print(f"Suma de residuos al cuadrado: {res[0]:.4f}")
19 print(f"Rango de la matriz de diseno: {rank}")
20 print(f"Valores singulares: {s}")

```

Listing 14: Ajuste polinomial experimental con `np.linalg.lstsq`

14. Número de Condición y Sensibilidad Numérica

14.1. Definición Formal y Propagación de Errores

El **número de condición** $\kappa(\mathbf{A})$ mide la sensibilidad de la solución del sistema lineal $\mathbf{Ax} = \mathbf{b}$ frente a pequeñas perturbaciones en los datos de entrada $\mathbf{b}$ o en la matriz $\mathbf{A}$:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\| = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})} \geq 1 \quad (25)$$

Teorema y Propiedad: Cota Fundamental de Propagación de Error

Si el término independiente se perturba en $\delta \mathbf{b}$, la variación relativa inducida en la solución $\mathbf{x}$ satisface rigurosamente:

$$\frac{\|\delta \mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A}) \frac{\|\delta \mathbf{b}\|}{\|\mathbf{b}\|} \quad (26)$$

Advertencia Numérica: Regla Práctica de Pérdida de Dígitos Decimales

Si $\kappa(\mathbf{A}) \approx 10^k$, se pueden perder hasta k dígitos decimales de precisión significativa en los cálculos. Una matriz con $\kappa(\mathbf{A}) \approx 1$ se denomina **bien condicionada** (por ejemplo, cualquier matriz ortogonal cumple $\kappa(\mathbf{Q}) = 1$); si $\kappa(\mathbf{A}) \geq 10^{14}$, la matriz se considera **numéricamente singular o mal condicionada** en aritmética de doble precisión.

```

1 import numpy as np
2
3 # Generacion de una matriz de Hilbert de dimension 7x7: H_ij = 1 / (i + j + 1)
4 n = 7
5 H = np.array([[1.0 / (i + j + 1) for j in range(n)] for i in range(n)])
6
7 cond_H = np.linalg.cond(H)
8 print(f"Numero de condicion kappa(H) de dimension {n}x{n}: {cond_H:.2e}")
9
10 # Solucion exacta arbitraria: x_exacto = [1, 1, 1, 1, 1, 1, 1]^T
11 x_exacto = np.ones(n)
12 b = H @ x_exacto
13
14 # Agregamos una micro-perturbacion de apenas 10^-10 a b
15 b_perturbado = b.copy()
16 b_perturbado[0] += 1e-10
17
18 x_recuperado = np.linalg.solve(H, b_perturbado)
19 error_relativo = np.linalg.norm(x_recuperado - x_exacto) / np.linalg.norm(x_exacto)
20 print(f"Perturbacion relativa en b: {1e-10 / np.linalg.norm(b):.2e}")
21 print(f>Error relativo amplificado en x: {error_relativo:.2e}")

```

Listing 15: Demostración del impacto del mal condicionamiento (Matriz de Hilbert)

15. Tabla Resumen de Funciones y Complejidad

Tabla 2: Compendio integral de funciones de `numpy.linalg`, restricciones y complejidad

Función NumPy	Operación / Significado	Requisitos	Complejidad	LAPACK
<code>det(A)</code>	Determinante escalar	$n \times n$	$\mathcal{O}(n^3)$	<code>dgetrf</code>
<code>slogdet(A)</code>	Signo y $\ln \det(A) $ numéricamente estable	$n \times n$	$\mathcal{O}(n^3)$	<code>dgetrf</code>
<code>inv(A)</code>	Inversa exacta $\mathbf{A}^{-1}$	$n \times n$, regular	$\mathcal{O}(2n^3)$	<code>dgetri</code>
<code>solve(A, b)</code>	Solución directa de $\mathbf{Ax} = \mathbf{b}$	$n \times n$, regular	$\mathcal{O}(\frac{2}{3}n^3)$	<code>dgesv</code>
<code>eig(A)</code>	Autovalores y autovectores en $\mathbb{C}$	$n \times n$	$\mathcal{O}(n^3)$	<code>dgeev</code>
<code>eigh(A)</code>	Autovalores reales de matriz simétrica	Simétrica/Hermítica	$\mathcal{O}(\frac{4}{3}n^3)$	<code>dsyevd</code>
<code>matrix_rank(A)</code>	Rango numérico con umbral SVD	$m \times n$	$\mathcal{O}(mn \min(m, n))$	<code>dgesdd</code>
<code>trace(A)</code>	Traza (suma de diagonal)	Cuadrada o rectangular	$\mathcal{O}(\min(m, n))$	BLAS 1
<code>norm(A, ord)</code>	Norma vectorial o matricial	Vector o Matriz	$\mathcal{O}(mn)$ o $\mathcal{O}(mn^2)$	<code>dlange</code>
<code>svd(A)</code>	Descomposición en Valores Singulares	$m \times n$ cualquiera	$\mathcal{O}(mn \min(m, n))$	<code>dgesdd</code>
<code>qr(A)</code>	Factorización ortogonal $\mathbf{A} = \mathbf{QR}$	$m \geq n$	$\mathcal{O}(2mn^2 - \frac{2}{3}n^3)$	<code>dgeqrf</code>
<code>cholesky(A)</code>	Factorización $\mathbf{A} = \mathbf{LL}^T$	Simétrica Def. Pos.	$\mathcal{O}(\frac{1}{3}n^3)$	<code>dpotrf</code>
<code>pinv(A)</code>	Pseudoinversa de Moore-Penrose	$m \times n$ cualquiera	$\mathcal{O}(mn \min(m, n))$	<code>dgesdd</code>
<code>lstsq(A, b)</code>	Mínimos cuadrados lineales	$m \geq n$	$\mathcal{O}(mn^2)$	<code>dgelsd</code>
<code>cond(A)</code>	Número de condición $\sigma_{\max}/\sigma_{\min}$	Matriz general	$\mathcal{O}(mn \min(m, n))$	<code>dgesdd</code>

16. Ejemplo Integrador Completo Multidisciplinar

El siguiente proyecto sintetiza los conceptos estudiados en un problema de ingeniería de datos y análisis de señales: calibración de un sistema de sensores correlacionados, filtrado de ruido espectral y estimación de parámetros óptimos.

```

1 import numpy as np
2
3 print("="*65)
4 print("LABORATORIO NUMERICO INTEGRADOR: PROCESAMIENTO DE SENSORES")
5 print("="*65)
6
7 # 1. Matriz de Covarianza Simetrica Definida Positiva (SPD)
8 Sigma = np.array([[4.0, 1.2, 0.8],
9                   [1.2, 3.0, 0.5],
10                  [0.8, 0.5, 2.0]])
11
12 # Verificacion de propiedades estructurales
13 det_S = np.linalg.det(Sigma)
14 cond_S = np.linalg.cond(Sigma)
15 rango_S = np.linalg.matrix_rank(Sigma)
16 traza_S = np.trace(Sigma)
17
18 print(f"Det: {det_S:.4f} | Rango: {rango_S} | Traza: {traza_S:.2f} | Cond: {cond_S:.2f}")
19
20 # 2. Descomposicion de Cholesky para generar 10,000 mediciones simuladas
21 L = np.linalg.cholesky(Sigma)
22 np.random.seed(2026)
23 ruido_blanco = np.random.randn(3, 10000)
24 sensores_correlacionados = L @ ruido_blanco
25
26 cov_muestra = np.cov(sensores_correlacionados)
27 print("\nCovarianza muestral simulada:\n", np.round(cov_muestra, 2))
28 print("Consistencia Cholesky:", np.allclose(Sigma, L @ L.T))
29
30 # 3. Analisis Espectral de Modos Principales (PCA manual via eig)
31 autovals, autovecs = np.linalg.eigh(Sigma)
32 print("\nAutovalores (energia por canal):", autovals)
33 print("Varianza explicada por el modo dominante:", (autovals[-1] /
    np.sum(autovals)) * 100, "%")

```

```

34
35 # 4. Ajuste y Calibracion de Sistema Lineal Ax = b
36 # Se desea calibrar una respuesta fisica con pesos conocidos
37 A_calib = np.array([[3.0, 1.0, 0.5],
38                    [1.0, 4.0, 1.0],
39                    [0.5, 1.0, 2.5]])
40 b_calib = np.array([5.5, 10.0, 6.5])
41
42 # Resolucion directa del sistema
43 x_calibrado = np.linalg.solve(A_calib, b_calib)
44 residuo_norma = np.linalg.norm(A_calib @ x_calibrado - b_calib)
45 print(f"\nVector calibrado x: {x_calibrado}")
46 print(f"Norma del residuo de calibracion: {residuo_norma:.2e}")
47
48 # 5. Filtrado de Ruido Matricial via SVD (Truncamiento de Rango)
49 U, s, Vt = np.linalg.svd(sensores_correlacionados[:, :100], full_matrices=False)
50 # Conservamos solo los 2 modos dominantes eliminando el ruido del 3er canal
51 s_filtrada = s.copy()
52 s_filtrada[2] = 0.0
53 sensores_filtrados = U @ np.diag(s_filtrada) @ Vt
54 print(f"Sensores filtrados (dimension): {sensores_filtrados.shape}")
55 print("Laboratorio completado satisfactoriamente.")

```

Listing 16: Pipeline completo integrando Cholesky, SVD, Solve, Cond y Verificaciones

17. Decálogo de Buenas Prácticas en Álgebra Lineal

1. **Priorizar el operador @ sobre bucles o np.dot:** En Python moderno (3,5+), el operador matricial @ (`__matmul__`) invoca directamente BLAS Nivel 3 y respeta semánticamente el producto de matrices, mejorando la legibilidad matemática.
2. **Jamás verificar igualdad flotante con ==:** Debido a las imprecisiones intrínsecas de IEEE 754, emplee siempre `np.allclose(A, B, rtol=1e-5, atol=1e-8)` o `np.isclose()`.
3. **No calcular inversas para resolver sistemas:** Emplee invariablemente `np.linalg.solve(A, b)` para sistemas cuadrados y `np.linalg.lstsq(A, b)` para sobredeterminados.
4. **Distinción entre eig y eigh:** Para matrices simétricas o hermiticas, el uso de `eigh` duplica la velocidad, garantiza valores propios estrictamente reales y asegura autovectores ortonormales.
5. **Monitorear el número de condición:** Antes de confiar en la salida de un modelo lineal crítico, evalúe `np.linalg.cond(A)`. Si supera 10^8 , considere técnicas de regularización (Tikhonov / Ridge Regression).
6. **Usar slogdet para evitar desbordamientos:** Al calcular probabilidades conjuntas o verosimilitudes con determinantes de matrices de covarianza de alta dimensión, evite `det(A)` y utilice el logaritmo de su determinante.
7. **Aprovechar la factorización de Cholesky:** En matrices de covarianza, optimización cuadrática o filtros de Kalman, `cholesky` reduce a la mitad el tiempo computacional respecto a LU y detecta inmediatamente si la matriz ha perdido positividad.
8. **Controlar copias de memoria innecesarias:** Preste atención a `np.ascontiguousarray()` si LAPACK reporta degradación de rendimiento con matrices generadas por transposiciones (*strided arrays*).
9. **Especificar rcond=None en lstsq:** En versiones recientes de NumPy, omitir este argumento emite una advertencia futura (*FutureWarning*). Especifíquelo explícitamente.

10. **Seleccionar el tipo de dato (dtype) adecuado:** Para cómputo general, `float64` ofrece 53 bits de mantisa ($\approx 15 - 17$ dígitos decimales). Para deep learning o grandes volúmenes de datos con memoria acotada, evalúe `float32`.

18. Conclusiones y Referencias Bibliográficas

El módulo `numpy.linalg` es la columna vertebral que posibilita la computación científica de alto rendimiento en el lenguaje Python. Su diseño conecta la abstracción del álgebra lineal teórica con el rendimiento de código compilado de bajo nivel a través de BLAS y LAPACK. El dominio de sus funciones, la comprensión de sus complejidades asintóticas y el discernimiento sobre la estabilidad numérica de cada descomposición matricial constituyen habilidades indispensables para cualquier profesional e investigador en ciencias exactas, computación e ingeniería.

Referencias

- [1] G. H. Golub y C. F. Van Loan, *Matrix Computations*, 4ta edición, The Johns Hopkins University Press, Baltimore, 2013.
- [2] L. N. Trefethen y D. Bau III, *Numerical Linear Algebra*, SIAM: Society for Industrial and Applied Mathematics, Filadelfia, 1997.
- [3] G. Strang, *Linear Algebra and Its Applications*, 4ta edición, Cengage Learning, 2006.
- [4] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2da edición, SIAM, 2002.
- [5] C. R. Harris et al., *Array programming with NumPy*, Nature, vol. 585, pp. 357–362, 2020. <https://doi.org/10.1038/s41586-020-2649-2>
- [6] E. Anderson et al., *LAPACK Users' Guide*, 3ra edición, SIAM, Filadelfia, 1999. <https://www.netlib.org/lapack/>
- [7] NumPy Community, *Linear algebra (numpy.linalg) Manual Reference*, <https://numpy.org/doc/stable/reference/routines.linalg.html>, 2026.